PROGRAMACIÓN EN PYTHON · SESIÓN 2 DOBLE

Estructuras de Control de Flujo

Toma de decisiones con if-elif-else, match-case e iteraciones limpias con for y while

UT 1 Cierre · UT 2 Arranque · Bloque de 2 horas (Teoría + 2 Talleres de código)

Selección: if / elif / else

Sin paréntesis obligatorios ni llaves. Los dos puntos (**:**) abren el bloque indentado.

elif: contracción directa de *else if*.

Rangos encadenados: 0 <= nota <= 10 válido sin duplicar la variable.

```
nota = 8.5

if nota ⩾ 9:
     calificacion = "Sobresaliente"
elif nota ⩾ 7:
     calificacion = "Notable"
elif nota ⩾ 5:
     calificacion = "Aprobado"
else:
     calificacion = "Suspenso"
```

Expresiones condicionales concisas

Operador ternario

Sintaxis natural: **valor_si** if condición else **valor_no**.

```
edad = 19
 estado = "Mayor" if edad ⩾ 18 else "Menor"
```

Evaluación de veracidad (Truthy)

Valores falsos directos: 0, "", [], None.

```
usuario = ""
 if not usuario:
     print("El campo no puede estar vacío")
```

Selección múltiple: match – case

Supera al clásico switch de Java. No requiere break entre casos.

case _: caso por defecto (equivalente al *default*).

case 200 | 201: permite agrupar varios valores con la barra vertical.

```
codigo_http = 404

match codigo_http:
      case 200 | 201:
          mensaje = "Petición correcta"
      case 404:
          mensaje = "Recurso no encontrado"
      case _:
          mensaje = "Estado desconocido"
```

Pattern Matching Avanzado en Python

Estructuras de control avanzadas y desempaquetado con la instrucción match-case

1. Switch-Case vs Match-Case

A diferencia del switch tradicional, match-case evalúa patrones complejos y extrae datos dinámicamente.

```
# Switch tradicional (valores)
match status:
      case 200:
            print("OK")
      case 404:
            print("Not Found")


# Pattern Matching (estructural)
match command:
      case ["quit"]:
            exit()
      case ["load", filename]:
            load_file(filename)
      case _:
            print(f"Error: comando
      '{command}' no reconocido.")
```

2. List & Tuple Unpacking

Permite capturar elementos específicos o secuencias usando comodines y operadores de desempaquetado.

```
# Desempaquetado de listas
match point:
      case (0, 0):
            print("Origen")
      case (0, y):
            print(f"Eje Y: {y}")
      case (x, y):
            print(f"Punto: {x},{y}")


# Captura de restos (*rest)
items = ["Mario", "Lucía", "Carlos"]
match items:
      case [first, *rest]:
            print(f"Primero: {first}")
            print(f"Resto: {rest}")

first: Almacena el valor "Mario" (tipo
str).
rest: Almacena la sublista ['Lucía',
'Carlos'] (tipo list)
```

3. Type Inspection & Filtering

Filtra por tipos de datos explícitos o aplica cláusulas guard (if) para comprobaciones avanzadas.

```
# Filtrado por tipo
match val:
      case int(x):
            print(“Es un entero”)
      case str(s):
            print(“Es un string)
      case _:
            print("Es de otro tipo")


# Guardias condicionales
match data:
      case n if n > 100:
            p("Valor alto")
      case _:
            p("Por defecto")
```

TALLER 1 · MANOS AL TECLADO (20 MIN)

Reto 1: Clasificador de billetes de tren

Crear archivo: 04_billetes.py

El programa solicita los datos del billete en una sola línea mediante un comando con argumentos separados por espacios:

- *Formato esperado:* TIPO_BILLETE EDAD [EXTRAS...]
- *Ejemplos válidos de entrada:*
  - TURISTA 16
  - PREFERENTE 68 WIFI
  - VIP 30 SILENCIO COMIDA SALA_VIP

El script debe parsear la entrada con .split() y evaluar toda la operación en un **único bloque match-case** que cubra:

1. **Patrón estructural con tupla/lista y guardas (if):**
   - Evaluar tipo de billete y edad simultáneamente mediante captura y comprobación de rangos numéricos.
   - *Descuento Joven:* edad < 18 aplica 25% de descuento.
   - *Descuento Senior:* edad ≥ 65 aplica 40% de descuento.
   - *Adulto (18–64):* tarifa base completa.
2. **Precios base por tarifa:**
   - TURISTA: 30 €
   - PREFERENTE: 50 €
   - VIP: 80 €
3. **Captura de extras con restos (*extras):**
   - En billetes PREFERENTE o VIP, el usuario puede añadir extras opcionales al final. El patrón debe capturarlos usando *extras y cobrar 10 € adicionales por cada extra solicitado.
4. **Validación y comodines:**
   - *Caso error de tarifa/edad:* Si la edad no es un número válido o es negativa, o la clase no existe, informar del error correspondiente.
   - *Caso comodín (_):* Capturar entradas que no cumplan el formato mínimo requerido.

Bucles for y la función range()

En Python el bucle for no tiene contador manual (i++); siempre recorre un iterable.

range(stop): desde 0 hasta stop-1.

range(start, stop): desde start hasta stop-1.

range(start, stop, step): define incremento o decremento.

```
# Cuenta atrás desde 5 hasta 1 con paso -1
for seg in range(5, 0, -1):
    print(f"Lanzamiento en {seg}...")


# Iterar directamente caracteres de un texto
for char in "PYTHON":
    print(char, end="-") # P-Y-T-H-O-N-
```

Bucles while y control de salto

`while`

Condición dinámica

Se ejecuta mientras la expresión sea True. Ideal para bucles interactivos y menús de consola.

`break / continue`

Interrupción y salto

break finaliza el bucle.
continue salta a la siguiente iteración inmediatamente.

`else en bucles`

¡Exclusivo de Python!

El bloque else se ejecuta solo si el bucle termina con normalidad (sin encontrar un break).

```
usuarios = ["Carlos", "Lucía", "Ana"]
buscado = "Pedro"

for u in usuarios:
  if u == buscado:
    print(f"¡Usuario {buscado} encontrado!")
    break  # Si lo encuentra, rompe el bucle
else:
  # Solo entra aquí si el bucle revisó toda la lista y NO hubo break
  print(f"El usuario {buscado} no existe en el sistema.")
```

TALLER 2 · MANOS AL TECLADO (25 MIN)

Reto 2: El juego del número secreto

Crear archivo: 05_numero_secreto.py

1. **Número aleatorio:** Usar import random; numero = random.randint(1, 50).
2. **Bucle de intentos:** Máximo 5 intentos para adivinar el número.
3. **Pistas:** Indicar si el número secreto es "Mayor" o "Menor".
4. **Cláusula else:** Si agota los intentos sin acertar, mostrar el número secreto usando.

EJEMPLO DE EJECUCIÓN

```
=== ADIVINA EL NÚMERO SECRETO (1 al 50) ===
Intento 1/5: 25
→ El número secreto es MAYOR.
Intento 2/5: 40
→ El número secreto es MENOR.
Intento 3/5: 30
→ El número secreto es MAYOR.
Intento 4/5: 35
→ El número secreto es MAYOR.
Intento 5/5: 37
→ El número secreto es MAYOR. Has agotado todos los
intentos. El número secreto era el 39.
```



¡Flujo de control dominado!

En la próxima sesión cerraremos la UT 2 con el control de excepciones (try / except / finally) y nos adentraremos en las Colecciones de datos.